

NetShield AI

Network Anomaly Detection & Threat Monitoring Platform

Project Documentation Report — Infosys Springboard Internship 7.0 Program

Submitted by: Vijay Kumar Reddy Ganapa

Mail id: ganapavijaykumarreddy1@gmail.com

GitHub Repo: <https://github.com/ganapavijaykumarreddy1/NetShieldAI> **Deployed Link:** <http://40.192.61.130>

1. Abstract / Executive Summary

NetShield AI is a full-stack network anomaly detection platform and Security Operations Center (SOC) dashboard developed as part of the Infosys Springboard internship program. The system captures live network traffic, extracts statistical flow-level features aligned with the CICIDS-2017 dataset format, and applies a supervised Machine Learning model (Random Forest Classifier) to distinguish benign traffic from malicious activity in real time. Detected anomalies are surfaced to security analysts through a centralized React-based dashboard that supports alert triage, incident aggregation, threat intelligence enrichment, and role-based access control.

2. Introduction & Problem Statement

Traditional network security tools, particularly signature-based firewalls and intrusion detection systems (IDS), operate by matching incoming traffic against a database of known attack patterns. While effective against previously catalogued threats, this approach has two structural limitations: it cannot detect novel or slightly modified attack variants that do not match an existing signature, and it requires constant manual updates to the signature database to remain relevant against emerging threats.

Anomaly-based detection using machine learning addresses this gap by learning the statistical characteristics of normal versus malicious traffic flows, rather than relying on fixed rules. A model trained on labeled traffic data can generalize to detect classes of behaviour — such as port scanning, denial-of-service floods, or brute-force attempts — even when the specific packet sequence has not been seen before. NetShield AI was built to demonstrate this approach in a practical, end-to-end system: from raw packet capture, through feature engineering and ML inference, to a usable analyst-facing interface for triage and response.

The project scope, as implemented, covers traffic sniffing and feature extraction, ML-based classification, an interactive SOC dashboard, incident management, threat intelligence enrichment, authentication with role-based access control, a controlled demonstration engine for safely showcasing detection behaviour, and basic reporting.

3. Core Modules & Features

3.1 Traffic Sniffing & Feature Extraction

The traffic module uses Scapy to capture packets from a network interface in an asynchronous background task, avoiding blocking of the main API event loop. Captured packets are handed to the feature extraction engine, which aggregates them into flows and computes 25 canonical statistical metrics consistent with the

CICIDS-2017 dataset format — including packet length distributions, flow duration, and byte/packet counts in each direction. This converts raw, unstructured packet data into a fixed-length numerical vector suitable for input to the ML model.

3.2 Machine Learning Detection Engine

A pre-trained Random Forest Classifier evaluates each extracted feature vector and outputs a classification — Normal Traffic or a specific attack category such as DDoS, DoS, Port Scanning, Brute Force, Bots, or Web Attacks — along with a computed risk score used to prioritize the resulting alert. The model is trained offline against CICIDS-2017-derived data, and the trained artifact is loaded at inference time rather than retrained live.

3.3 Interactive SOC Dashboard

The React frontend provides analysts with a real-time operational view of the network, built from reusable components (metric cards, sidebar, navbar) and Recharts-based visualizations. The dashboard surfaces current traffic statistics, active alerts, and a logical threat map summarizing where detected activity is originating from, giving analysts situational awareness without needing to inspect raw logs.

3.4 Incident Management

Rather than presenting every raw alert in isolation, the backend automatically aggregates related alerts into Incidents based on shared source/destination IP addresses and time proximity. This reduces alert fatigue and gives analysts a consolidated view of a single ongoing event. From the dashboard, analysts can assign incidents to themselves or team members, mark them as under investigation, and close them once resolved.

3.5 Threat Intelligence Module

A dedicated threat intelligence store maintains seeded reference data on known malicious IP addresses, threat actors, and campaigns. When an alert or incident involves an IP present in this store, the record is used to enrich the alert with contextual information, helping analysts understand whether an event is linked to previously known malicious infrastructure.

3.6 Role-Based Access Control (RBAC)

Authentication is implemented using JWT tokens, with secure password hashing and token issuance/validation on the backend. The system supports three roles — Administrator, SOC manager and Analyst — with route-level and UI-level access differentiated accordingly (e.g., administrative functions such as user management are restricted to the Administrator role). A seed utility provisions a default administrator account on first deployment.

3.7 Built-in Demonstration Engine

Because triggering real malware or live attacks against a network for evaluation purposes is impractical and risky, NetShield AI includes a controlled Demo Mode. This allows an evaluator to trigger synthetic, safe traffic patterns simulating specific attack types — such as Port Scanning or a DoS/DDoS flood — and observe the full detection pipeline react in real time, from feature extraction through classification to alert generation on the dashboard. A companion reports live system performance figures such as packets processed per second.

3.8 Reporting

The reporting module allows analysts to export summaries of incidents and traffic activity for record-keeping or offline review, generating downloadable report files from the aggregated database records.

4. Machine Learning Methodology

4.1 Model Choice

NetShield AI uses a Random Forest Classifier (Scikit-Learn) as its core detection model. Random Forest was selected as a supervised ensemble method that combines multiple decision trees to reduce overfitting relative to a single tree, handles the mixed-scale numerical features typical of network flow data reasonably well, and provides fast inference — a requirement for near real-time classification of live traffic.

4.2 Feature Pipeline

Training and inference rely on the same 25-feature schema derived from the CICIDS-2017 dataset format. During training, the model is fit against labeled flow records covering Normal Traffic and known attack categories present in the dataset. During inference, the live feature extraction engine produces feature vectors in the identical schema, which are passed to the loaded model for classification, ensuring consistency between the training and serving representations of traffic.

4.3 Output and Risk Scoring

Each classified flow produces a predicted label (Normal Traffic or a specific attack type, e.g. DDoS, DoS, Port Scanning, Brute Force, Bots, or Web Attacks) together with a risk score used by the alerting logic to prioritize the resulting record. Higher-risk classifications surface more prominently on the SOC dashboard and are more likely to be aggregated into an Incident requiring analyst attention.

4.4 Evaluation Considerations

Model quality for this class of problem is typically assessed using standard classification metrics — accuracy, precision, recall, and F1-score — alongside security-specific measures such as false positive rate and threat detection rate, since an operationally useful detector must balance catching genuine attacks against avoiding excessive false alarms that would overwhelm analysts.

5. Model Evaluation & Results

The trained Random Forest Classifier was evaluated on a held-out test set of 5,04,151 samples (20% of total dataset) drawn from the CICIDS-2017 dataset, spanning Normal Traffic and six attack categories. The model achieved an overall accuracy of 99.71% and a weighted F1-score of 0.9971. The complete per-class classification report produced by the evaluation pipeline is reproduced below.

Class	Precision	Recall	F1-Score	Support
Bots	0.69	0.49	0.57	389
Brute Force	0.99	0.99	0.99	1,830
DDoS	1.00	1.00	1.00	25,603
DoS	1.00	0.99	1.00	38,749

Class	Precision	Recall	F1-Score	Support
Normal Traffic	1.00	1.00	1.00	419,012
Port Scanning	0.99	1.00	0.99	18,139
Web Attacks	0.92	0.21	0.34	429
Macro Avg	0.94	0.81	0.84	504,151
Weighted Avg	1.00	1.00	1.00	504,151

5.1 Analysis of Results

The model performs near-perfectly (0.99–1.00 precision/recall) on the high-volume classes — Normal Traffic, DDoS, DoS, and Port Scanning — since these attacks produce distinctive flow-level signatures well captured by the feature set and well represented in training (18,139–419,012 samples each).

Bots and Web Attacks perform weaker. Web Attacks has high precision (0.92) but low recall (0.21), missing most real instances, likely due to limited training samples (389–429) and because web-layer attacks often show up in encrypted payload content rather than flow-level statistics. Bots shows a similar, milder gap (precision 0.69, recall 0.49).

The weighted averages (1.00) are skewed by the majority classes; the macro average (F1-score 0.84) better reflects the weaker minority-class performance. Overall, the model is strong on volumetric attacks, while low-volume, application-layer attacks would benefit from further work like oversampling or payload-aware features.

6. Deployment & Infrastructure

NetShield AI is fully containerized using Docker and orchestrated with Docker Compose, with the frontend, backend, and PostgreSQL database running as separate services. The platform has been successfully deployed to a live AWS EC2 cloud server (at <http://40.192.61.130>) for public access. In this cloud environment, the containerized backend utilizes elevated network privileges to hook directly into the AWS server's interface, allowing the AI pipeline to actively sniff and analyze 100% of the real incoming and outgoing public internet traffic of that AWS server in real-time.

7. Total Workflow

Raw PCAP / Simulated Flow → Canonical Feature Extractor (25 Features) → StandardScaler Normalization → Random Forest Ensemble Classifier → Probability Array & Multi-Class Prediction → Threat Risk Scoring

8. Conclusion

NetShield AI demonstrates a complete pipeline for ML-driven network anomaly detection, from raw packet capture through feature extraction, classification, and analyst-facing incident response, packaged as a containerized, independently deployable system. The project integrates network programming (Scapy), applied machine learning (Scikit-Learn Random Forest on CICIDS-2017-style features), backend API design (FastAPI, SQLAlchemy, PostgreSQL), and frontend dashboard engineering (React, Recharts) into a single working SOC-style tool, along with a purpose-built demonstration engine that allows the detection pipeline to be safely evaluated without real attack traffic.